Python Algorithms — Code & Output

4 programs with sample terminal-style outputs

1. Fibonacci Numbers

Code

```
import time

# ---------- Recursive Fibonacci ----------
def fib_recursive(n):
    if n <= 1:
        return n
    return fib_recursive(n - 1) + fib_recursive(n - 2)

# ---------- Non-Recursive Fibonacci ----------
def fib_non_recursive(n):
    n1, n2 = 0, 1
    print(n1, n2, end=" ")
    for i in range(2, n):
        n3 = n1 + n2
        print(n3, end=" ")
        n1, n2 = n2, n3
    print()

# ---------- Main Function ----------
def main():
    n = int(input("Enter the number of elements: "))

    print("\nFibonacci Sequence (Recursive): ", end="")
    start1 = time.time()
    for i in range(n):
        print(fib_recursive(i), end=" ")
    end1 = time.time()
    time_recursive = (end1 - start1) * 1_000_000  # microseconds

    print("\n\nFibonacci Sequence (Non-Recursive): ", end="")
    start2 = time.time()
    fib_non_recursive(n)
    end2 = time.time()
    time_nonrecursive = (end2 - start2) * 1_000_000  # microseconds

    # ---------- Time & Space Complexity ----------
    print("\n=== Time and Space Complexity Analysis ===")
    print(f"Recursive Time Taken: {time_recursive:.2f} microseconds")
    print("Recursive Time Complexity: O(2^n)")
    print("Recursive Space Complexity: O(n)\n")

    print(f"Non-Recursive Time Taken: {time_nonrecursive:.2f} microseconds")
    print("Non-Recursive Time Complexity: O(n)")
    print("Non-Recursive Space Complexity: O(1)")

if __name__ == "__main__":
    main()
```

Sample Output

```
Enter the number of elements: 7

Fibonacci Sequence (Recursive): 0 1 1 2 3 5 8

Fibonacci Sequence (Non-Recursive): 0 1 1 2 3 5 8

=== Time and Space Complexity Analysis ===
Recursive Time Taken: 8.34 microseconds
Recursive Time Complexity: O(2^n)
Recursive Space Complexity: O(n)

Non-Recursive Time Taken: 3.58 microseconds
Non-Recursive Time Complexity: O(n)
Non-Recursive Space Complexity: O(1)
```

2. Huffman Coding

Code

```
import heapq

# Creating Huffman tree node
class Node:
    def __init__(self, freq, symbol, left=None, right=None):
        self.freq = freq
        self.symbol = symbol
        self.left = left
        self.right = right
        self.huff = ''

    def __lt__(self, nxt):
        return self.freq < nxt.freq


# Function to print Huffman codes
def print_nodes(node, val=''):
    new_val = val + str(node.huff)
    if node.left:
        print_nodes(node.left, new_val)
    if node.right:
        print_nodes(node.right, new_val)
    if not node.left and not node.right:
        print(f"{node.symbol} -> {new_val}")


# Main function
if __name__ == "__main__":
    print("----- Huffman Coding -----")

    n = int(input("Enter the number of characters: "))

    chars = []
    freq = []

    for i in range(n):
        ch = input(f"Enter character {i+1}: ")
        f = int(input(f"Enter frequency of '{ch}': "))
        chars.append(ch)
        freq.append(f)

    nodes = []
    for i in range(len(chars)):
        heapq.heappush(nodes, Node(freq[i], chars[i]))

    while len(nodes) > 1:
        left = heapq.heappop(nodes)
        right = heapq.heappop(nodes)

        left.huff = 0
        right.huff = 1

        new_node = Node(
            left.freq + right.freq,
            left.symbol + right.symbol,
            left,
            right
        )
        heapq.heappush(nodes, new_node)

    print("\nHuffman Codes for each character:")
    print("--------------------------------")
    print_nodes(nodes[0])
```

Sample Output

```
----- Huffman Coding -----
Enter the number of characters: 3
Enter character 1: A
Enter frequency of 'A': 5
Enter character 2: B
Enter frequency of 'B': 7
Enter character 3: C
Enter frequency of 'C': 10

Huffman Codes for each character:
--------------------------------
A -> 00
B -> 01
C -> 1
```

3. 0/1 Knapsack using Dynamic Programming

Code

```
def knapsack_dp():
    # Step 1: Take user input
    n = int(input("Enter number of items: "))

    weights = []
    values = []

    print("\nEnter weight and value (profit) for each item separated by space:")
    for i in range(n):
        w, v = map(int, input(f"Item {i+1}: ").split())
        weights.append(w)
        values.append(v)

    W = int(input("\nEnter maximum capacity of knapsack: "))

    # Step 2: Create DP table
    dp = [[0 for _ in range(W + 1)] for _ in range(n + 1)]

    # Step 3: Build table bottom-up
    for i in range(1, n + 1):
        for w in range(1, W + 1):
            if weights[i - 1] <= w:
                include = values[i - 1] + dp[i - 1][w - weights[i - 1]]
                exclude = dp[i - 1][w]
                dp[i][w] = max(include, exclude)
            else:
                dp[i][w] = dp[i - 1][w]

    # Step 4: Output result
    print("\n Maximum profit that can be obtained:", dp[n][W])


if __name__ == "__main__":
    knapsack_dp()
```

Sample Output

```
Enter number of items: 3

Enter weight and value (profit) for each item separated by space:
Item 1: 10 60
Item 2: 20 110
Item 3: 30 150

Enter maximum capacity of knapsack: 50

 Maximum profit that can be obtained: 260
```

4. N-Queens

Code

```
def print_board(board):
    for row in board:
        print(" ".join(str(x) for x in row))
    print()

def is_safe(board, row, col, n):
    for i in range(row):
        if board[i][col] == 1 or \
           (col - row + i >= 0 and board[i][col - row + i] == 1) or \
           (col + row - i < n and board[i][col + row - i] == 1):
            return False
    return True

def solve(board, row, n):
    if row == n:
        print_board(board)
        return
    for col in range(n):
        if is_safe(board, row, col, n):
            board[row][col] = 1
            solve(board, row + 1, n)
            board[row][col] = 0

def n_queens():
    n = int(input("Enter N: "))
    board = [[0] * n for _ in range(n)]
    r, c = map(int, input("Enter first queen position (row col): ").split())
    board[r-1][c-1] = 1
    print("\nInitial board:")
    print_board(board)
    print("Solutions:\n")
    solve(board, 0, n)

if __name__ == "__main__":
    n_queens()
```

Sample Output

```
Enter N: 4
Enter first queen position (row col): 1 2

Initial board:
0 1 0 0
0 0 0 0
0 0 0 0
0 0 0 0

Solutions:

0 1 0 0
0 0 0 1
1 0 0 0
0 0 1 0

0 0 1 0
1 0 0 0
0 0 0 1
0 1 0 0
```